Product Requirement Document

Project Title: InternSetu AI

AI-Based Smart Allocation Engine for PM Internship Scheme

---

1. Project Overview

1.1 Project Idea

InternSetu AI is a smart internship allocation platform designed for the PM Internship Scheme. The platform uses AI/ML-based matching to connect candidates with suitable internship opportunities based on their skills, qualification, location preference, sector interest, company requirements, internship capacity, and fairness-related policy goals.

The system will include three dashboards:

1. Candidate Dashboard
2. Company / Employer Dashboard
3. Government / Admin Dashboard

The main objective is to make internship allocation faster, fairer, transparent, and explainable.

---

2. Problem Statement

The PM Internship Scheme provides students and young candidates with industry exposure through structured internships. However, when thousands or lakhs of candidates apply for limited internship opportunities, manual selection becomes slow, biased, and inefficient.

Candidates often struggle to find internships that match their skills, qualification, location, and career interests. Companies struggle to shortlist the right candidates from a large number of applications. Government authorities need to ensure fair representation across rural areas, aspirational districts, social categories, and different sectors.

Therefore, there is a need for an AI-based smart allocation system that can automatically recommend suitable internships to candidates, rank candidates for companies, and help government/admin users monitor fairness, capacity utilization, and transparency.

---

3. Proposed Solution

InternSetu AI will solve this problem by building a smart matchmaking engine that matches candidates with internships using a weighted scoring model.

The system will consider:

- Candidate skills
- Required internship skills
- Candidate qualification
- Internship eligibility criteria
- Candidate location preference
- Internship location
- Sector interest
- Company sector
- Internship capacity
- Past participation
- Rural/urban representation
- Aspirational district representation
- Social category fairness if applicable by policy

The system will also provide an explanation for every recommendation.

Example:

A candidate may see:

“Recommended because your skills match 85%, your qualification is eligible, the internship is in your preferred state, and your sector interest matches the role.”

---

4. Product Vision

To build a fair, explainable, and scalable AI-based internship allocation platform that helps candidates find suitable internships, helps companies find relevant interns, and helps government authorities monitor transparent and inclusive allocation.

---

5. Product Goals

5.1 Primary Goals

- Build AI-based internship recommendation system.
- Create separate dashboards for candidate, company, and admin.
- Generate match score between candidate and internship.
- Provide explainable recommendation.

- Track application status.
- Monitor fairness and capacity from admin dashboard.
- Create a working full-stack prototype.

5.2 Secondary Goals

- Improve trust in internship allocation.
- Reduce manual shortlisting work for companies.
- Support rural and underrepresented candidates.
- Improve transparency in government-level internship allocation.
- Build a scalable project that can be shown in hackathons and resume.

---

6. Target Users

6.1 Candidate

A student or youth who wants to apply for internships.

Candidate tasks:

- Register
- Complete profile
- Add skills
- Add qualification
- Select sector interest
- Select preferred location
- View recommended internships
- Apply to internships
- Track application status

6.2 Company / Employer

A company that wants to offer internship opportunities.

Company tasks:

- Register company
- Create internship post
- Add required skills
- Add qualification requirement
- Add location
- Add capacity
- View AI-ranked candidates
- Shortlist candidates
- Select/reject/waitlist candidates

- Track capacity utilization

6.3 Government / Admin

Government authority or scheme admin responsible for monitoring the allocation process.

Admin tasks:

- View overall dashboard
- Track total candidates
- Track total companies
- Track total internships
- Monitor district-wise allocation
- Monitor category-wise representation
- Monitor rural/urban participation
- Track company capacity utilization
- View AI audit logs
- Update matching policy weights

---

7. User Stories

7.1 Candidate User Stories

User Story 1: Candidate Registration

As a candidate,
I want to register on the platform,
so that I can create my internship profile.

Acceptance Criteria:

- Candidate can create account.
- Candidate can login.
- Candidate can access candidate dashboard.

User Story 2: Candidate Profile Creation

As a candidate,
I want to add my personal, educational, skill, and preference details,
so that the system can recommend suitable internships.

Acceptance Criteria:

- Candidate can add name, age, district, state, qualification, skills, sector interest, and location preference.
- Candidate profile is saved in database.
- Candidate can update profile later.

User Story 3: Internship Recommendation

As a candidate,
I want to see internships matching my skills and preferences,
so that I can apply to the most suitable opportunities.

Acceptance Criteria:

- Candidate can view top recommended internships.
- Each internship shows match score.
- Each internship shows company name, location, duration, role, and required skills.

User Story 4: Explainable Recommendation

As a candidate,
I want to know why an internship is recommended to me,
so that I can trust the system.

Acceptance Criteria:

- Every recommended internship shows explanation.
- Explanation includes skill match, qualification match, location match, sector match, and fairness factors if applicable.

User Story 5: Apply to Internship

As a candidate,
I want to apply to a recommended internship,
so that the company can review my profile.

Acceptance Criteria:

- Candidate can click Apply.
- Application status becomes Applied.
- Company can see the application.

User Story 6: Track Application Status

As a candidate,
I want to track my application status,
so that I know whether I am shortlisted, selected, rejected, or waitlisted.

Acceptance Criteria:

- Candidate can see application status.
- Status updates when company takes action.

7.2 Company User Stories

User Story 1: Company Registration

As a company,
I want to register and create a company profile,
so that I can post internships.

Acceptance Criteria:

- Company can register.
- Company can login.
- Company can update company details.

User Story 2: Create Internship

As a company,
I want to create internship opportunities,
so that candidates can be matched with them.

Acceptance Criteria:

- Company can add internship title, sector, required skills, qualification, location, duration, stipend, and capacity.
- Internship is saved in database.
- Internship becomes visible to matching engine.

User Story 3: View AI-Ranked Candidates

As a company,
I want to view ranked candidates for each internship,
so that I can shortlist suitable candidates faster.

Acceptance Criteria:

- Company can see candidate list sorted by match score.
- Each candidate has match percentage.
- Each candidate has explanation.

User Story 4: Shortlist Candidate

As a company,
I want to shortlist suitable candidates,
so that I can move them to the next stage.

Acceptance Criteria:

- Company can shortlist candidate.
- Candidate status changes to Shortlisted.
- Candidate can see updated status.

User Story 5: Select / Reject / Waitlist Candidate

As a company,
I want to select, reject, or waitlist candidates,
so that I can manage internship seats properly.

Acceptance Criteria:

- Company can change candidate status.
- Selected candidates reduce available capacity.
- Rejected candidates get rejection status.
- Waitlisted candidates are stored separately.

7.3 Government / Admin User Stories

User Story 1: Admin Dashboard

As an admin,
I want to see overall system analytics,
so that I can monitor the internship allocation process.

Acceptance Criteria:

- Admin can see total candidates.
- Admin can see total companies.

- Admin can see total internships.
- Admin can see total applications.
- Admin can see selected candidates and remaining seats.

User Story 2: District-Wise Allocation

As an admin,
I want to see district-wise allocation,
so that I can check whether rural and aspirational districts are represented.

Acceptance Criteria:

- Admin can view district-wise candidate count.
- Admin can view district-wise selected candidate count.
- Data is shown using charts or tables.

User Story 3: Category-Wise Representation

As an admin,
I want to monitor category-wise representation,
so that allocation remains inclusive.

Acceptance Criteria:

- Admin can view representation by category.
- Admin can view rural/urban distribution.
- Admin can identify underrepresented groups.

User Story 4: Capacity Utilization

As an admin,
I want to monitor company-wise capacity utilization,
so that internship seats are not wasted.

Acceptance Criteria:

- Admin can view company capacity.
- Admin can view selected candidates per company.
- Admin can view remaining seats.

User Story 5: AI Audit Logs

As an admin,
I want to view AI recommendation logs,
so that every decision can be reviewed.

Acceptance Criteria:

- Every match result is stored.
- Match score is stored.
- Explanation is stored.
- Status change history is stored.

User Story 6: Policy Weight Configuration

As an admin,
I want to update matching weights,
so that the system can adapt to new policy requirements.

Acceptance Criteria:

- Admin can view current weights.
- Admin can update weights.
- Updated weights affect future matching.

---

8. Key Features

8.1 Candidate Dashboard Features

- Candidate registration and login
- Candidate profile management
- Skill selection
- Qualification details
- Location preference
- Sector interest
- Recommended internships
- Match score card
- Explanation box
- Apply button
- Application status tracking

8.2 Company Dashboard Features

- Company registration and login
- Company profile
- Create internship post
- Internship capacity management
- Required skill configuration
- Candidate shortlist
- Candidate match score
- Candidate explanation
- Shortlist/select/reject/waitlist actions

8.3 Admin Dashboard Features

- Overall analytics
- Candidate analytics
- Company analytics
- Internship analytics
- District-wise allocation
- Category-wise representation
- Rural/urban representation
- Capacity utilization
- AI audit logs
- Policy weight settings

8.4 AI Matching Engine Features

- Eligibility filtering
- Skill matching
- Qualification matching
- Location matching
- Sector matching
- Fairness score calculation
- Capacity-aware ranking
- Final match score generation
- Explanation generation

---

9. Matching Algorithm

9.1 Matching Score Formula

The system will use a weighted scoring formula:

Final Score =
Skill Match Score × 0.35

- Qualification Score × 0.20
- Location Score × 0.20
- Sector Interest Score × 0.15
- Fairness Score × 0.10

9.2 Score Components

Skill Match Score

Compares candidate skills with internship required skills.

Example:

Candidate skills:

- Python
- Excel
- SQL

Required skills:

- Python
- Excel
- Data Analysis

Matched skills:

- Python
- Excel

Skill Score = 2 / 3 = 66.6%

Qualification Score

Checks whether candidate qualification matches internship requirement.

Example:

- Eligible = 1
- Not eligible = 0

Location Score

Checks candidate location preference and internship location.

Example:

- Same district = 1.0
- Same state = 0.75
- Different state but willing to relocate = 0.4
- Not preferred = 0

Sector Score

Checks whether candidate interest matches internship sector.

Example:

- Exact match = 1.0
- Related sector = 0.5
- No match = 0

Fairness Score

Supports representation goals.

Possible factors:

- Rural candidate
- Aspirational district candidate
- Underrepresented category
- PwD candidate
- First-time participant

Important: Fairness score should be used carefully as a policy-aware boost, not as unfair discrimination.

---

10. System Workflow

10.1 Candidate Flow

1. Candidate registers.
2. Candidate completes profile.
3. Candidate adds skills and preferences.
4. System runs matching engine.

5. Candidate sees top internship recommendations.
6. Candidate checks explanation.
7. Candidate applies.
8. Candidate tracks status.

10.2 Company Flow

1. Company registers.
2. Company creates internship post.
3. System matches candidates with internship.
4. Company views ranked candidates.
5. Company checks match explanation.
6. Company shortlists candidate.
7. Company selects/rejects/waitlists candidate.
8. Capacity gets updated.

10.3 Admin Flow

1. Admin logs in.
2. Admin views overall analytics.
3. Admin checks district-wise allocation.
4. Admin checks representation analytics.
5. Admin checks capacity utilization.
6. Admin views AI audit logs.
7. Admin updates policy weights if required.

---

11. Functional Requirements

11.1 Authentication

- User should be able to login.
- System should support three roles:
- Candidate
- Company
- Admin

For MVP, role-based demo login can be used.

11.2 Candidate Management

- Create candidate profile.
- Read candidate profile.
- Update candidate profile.

- Store skills, qualification, district, category, sector interest, and preferences.

11.3 Company Management

- Create company profile.
- Update company profile.
- Add company sector and location.

11.4 Internship Management

- Company can create internship.
- Company can update internship.
- Company can view posted internships.
- Internship should include title, skills, qualification, location, duration, stipend, and capacity.

11.5 Application Management

- Candidate can apply to internship.
- Company can update application status.
- Candidate can track status.
- Admin can view application analytics.

11.6 Matching Engine

- System should calculate match score.
- System should rank internships for candidate.
- System should rank candidates for company.
- System should generate explanation.
- System should consider capacity.

11.7 Admin Analytics

- Total candidates
- Total companies
- Total internships
- Total applications
- Selected candidates
- Remaining seats
- District-wise allocation
- Category-wise representation
- Company capacity utilization

- Fairness score
- Audit logs

---

12. Non-Functional Requirements

12.1 Performance

- Recommendation API should respond within 1–2 seconds for MVP.
- Dashboard data should load within 2–3 seconds.
- Matching should work smoothly for sample datasets.

12.2 Scalability

MVP can use SQLite, but production should support:

- PostgreSQL
- Redis caching
- Background job queues
- Batch recommendation generation
- Cloud deployment
- Microservice architecture

12.3 Security

- Passwords should be hashed.
- Role-based access should be implemented.
- Candidate should not access company/admin pages.
- Company should not access admin pages.
- APIs should validate inputs.
- Sensitive data should not be exposed publicly.

12.4 Privacy

The system should collect only required data.

Privacy principles:

- Consent-based data collection
- Purpose limitation
- Data minimization
- Secure storage
- Controlled access

- Auditability

12.5 Explainability

Every recommendation should include:

- Skill match explanation
- Qualification explanation
- Location explanation
- Sector explanation
- Fairness explanation if used

12.6 Reliability

- APIs should handle errors properly.
- System should show proper error messages.
- Dashboard should not crash if data is empty.
- Matching engine should handle missing fields.

12.7 Usability

- UI should be simple and clean.
- Candidate dashboard should be mobile-friendly.
- Buttons should be clear.
- Charts should be easy to understand.
- Language should be simple.

---

13. Tech Stack

13.1 Recommended Hackathon Stack

Frontend:

- React.js
- Tailwind CSS
- React Router
- Axios
- Recharts

Backend:

- FastAPI

- Pydantic
- SQLAlchemy

Database:

- SQLite for local development
- PostgreSQL for production

AI/ML:

- Python
- Pandas
- Scikit-learn
- Custom matching algorithm

Deployment:

- Frontend: Vercel
- Backend: Render
- Database: Supabase/PostgreSQL

---

14. Database Design

14.1 Candidate Table

Fields:

- candidate_id
- name
- email
- password_hash
- age
- gender
- category
- rural_or_urban
- district
- state
- qualification
- skills
- sector_interest
- location_preference
- willing_to_relocate
- past_participation
- created_at

14.2 Company Table

Fields:

- company_id
- company_name
- email
- password_hash
- sector
- location
- district
- state
- contact_person
- description
- created_at

14.3 Internship Table

Fields:

- internship_id
- company_id
- title
- sector
- required_skills
- required_qualification
- location
- district
- state
- duration
- stipend
- capacity
- selected_count
- created_at

14.4 Application Table

Fields:

- application_id
- candidate_id
- internship_id
- status
- applied_at
- updated_at

Status values:

- Recommended
- Applied
- Shortlisted
- Selected
- Rejected
- Waitlisted

14.5 Match Result Table

Fields:

- match_id
- candidate_id
- internship_id
- skill_score
- qualification_score
- location_score
- sector_score
- fairness_score
- final_score
- explanation
- created_at

14.6 Audit Log Table

Fields:

- audit_id
- user_role
- user_id
- action
- description
- timestamp

---

15. API Requirements

15.1 Candidate APIs

POST /candidates
Create candidate profile.

GET /candidates
Get all candidates.

GET /candidates/{candidate_id}
Get candidate by ID.

PUT /candidates/{candidate_id}
Update candidate profile.

GET /candidates/{candidate_id}/recommendations
Get recommended internships.

15.2 Company APIs

POST /companies
Create company profile.

GET /companies
Get all companies.

GET /companies/{company_id}
Get company details.

PUT /companies/{company_id}
Update company profile.

15.3 Internship APIs

POST /internships
Create internship.

GET /internships
Get all internships.

GET /internships/{internship_id}
Get internship details.

GET /companies/{company_id}/internships
Get internships posted by company.

15.4 Matching APIs

POST /match/candidate/{candidate_id}
Generate internship recommendations for candidate.

POST /match/internship/{internship_id}
Generate ranked candidates for internship.

GET /match/results
Get match results.

15.5 Application APIs

POST /applications
Candidate applies for internship.

GET /applications/candidate/{candidate_id}
Get candidate applications.

GET /applications/company/{company_id}
Get company applications.

PUT /applications/{application_id}/status
Update application status.

15.6 Admin APIs

GET /admin/overview
Get total candidates, companies, internships, applications.

GET /admin/district-analytics
Get district-wise allocation.

GET /admin/category-analytics
Get category-wise representation.

GET /admin/capacity-analytics
Get company capacity utilization.

GET /admin/audit-logs
Get audit logs.

PUT /admin/policy-weights
Update matching weights.

16. Frontend Pages

16.1 Public Pages

- Landing Page
- Login Page
- Register Page
- Role Selection Page

16.2 Candidate Pages

- Candidate Dashboard
- Candidate Profile
- Recommended Internships
- Internship Details
- Application Status

16.3 Company Pages

- Company Dashboard
- Company Profile
- Create Internship
- My Internships
- Candidate Shortlist
- Selected Candidates
- Capacity Utilization

16.4 Admin Pages

- Admin Overview
- Candidate Analytics
- Company Analytics
- District-Wise Allocation
- Category-Wise Representation
- Capacity Utilization
- AI Audit Logs
- Policy Settings

---

17. MVP Scope

The MVP should focus only on the most important features.

17.1 MVP Features

Candidate:

- Create profile
- View recommended internships
- See match score
- See explanation
- Apply to internship
- Track status

Company:

- Create internship
- View ranked candidates
- Shortlist/select/reject candidate
- Track capacity

Admin:

- View overview analytics
- View district-wise allocation
- View category-wise representation
- View capacity utilization
- View audit logs

AI Engine:

- Rule-based eligibility filter
- Weighted scoring model
- Fairness boost
- Explanation generation

17.2 Features Not Required in MVP

These can be added later:

- Real Aadhaar/DigiLocker integration
- Payment or stipend management
- Real-time notifications
- Resume parser
- Advanced ML model training
- Mobile app
- Full multilingual support
- Video interview module
- Chatbot support

---

18. Future Scope

After MVP, the project can be improved with:

- Resume parsing using NLP
- Semantic skill matching using embeddings
- DigiLocker document verification
- Aadhaar-based identity verification if officially permitted
- Multilingual interface
- Candidate skill-gap recommendation
- Employer feedback learning
- Real-time notification system
- Mobile app
- Advanced fairness monitoring
- ML model performance dashboard
- Large-scale batch allocation engine
- Integration with PMIS, AICTE, NCS, and Skill India ecosystem

---

19. Development Timeline

19.1 Full Implementation Timeline

Recommended duration: 35 days

Daily effort: 3–5 hours

Phase 1: Planning and Setup

Duration: 2 days

Tasks:

- Finalize project idea.
- Finalize features.
- Finalize tech stack.
- Create GitHub repository.
- Create frontend and backend folders.
- Prepare sample data.
- Design database schema.

Deliverables:

- Project structure
- README draft

- Database plan
- Sample candidate and internship data

Phase 2: Backend Development

Duration: 5 days

Tasks:

- Setup FastAPI.
- Setup database connection.
- Create models.
- Create schemas.
- Create routes.
- Build candidate APIs.
- Build company APIs.
- Build internship APIs.
- Build application APIs.

Deliverables:

- Working backend
- CRUD APIs
- Database connected

Phase 3: AI Matching Engine

Duration: 6 days

Tasks:

- Build eligibility filter.
- Build skill scoring.
- Build qualification scoring.
- Build location scoring.
- Build sector scoring.
- Build fairness scoring.
- Build final match score.
- Build explanation generator.

Deliverables:

- Working AI matching engine
- Candidate-to-internship recommendation
- Internship-to-candidate ranking

- Explainable score output

Phase 4: Frontend Development

Duration: 8 days

Tasks:

- Setup React project.
- Setup Tailwind CSS.
- Setup routing.
- Build landing page.
- Build candidate dashboard.
- Build company dashboard.
- Build admin dashboard.
- Build reusable components.

Deliverables:

- Working frontend screens
- Candidate dashboard
- Company dashboard
- Admin dashboard

Phase 5: Integration

Duration: 5 days

Tasks:

- Connect frontend with backend.
- Connect candidate dashboard APIs.
- Connect company dashboard APIs.
- Connect admin dashboard APIs.
- Implement application status flow.
- Implement audit logs.

Deliverables:

- End-to-end working project
- Candidate → Company → Admin flow working

Phase 6: Testing and UI Polish

Duration: 5 days

Tasks:

- Test backend APIs.
- Test frontend pages.
- Test matching engine.
- Fix bugs.
- Improve UI.
- Add loading states.
- Add empty states.
- Add validation.

Deliverables:

- Stable MVP
- Clean UI
- Tested project

Phase 7: Deployment and Documentation

Duration: 4 days

Tasks:

- Write README.
- Add screenshots.
- Prepare architecture diagram.
- Deploy frontend.
- Deploy backend.
- Record demo video.
- Prepare pitch deck.

Deliverables:

- GitHub repository
- Live frontend link
- Live backend link
- Demo video
- Project documentation

---

20. File Structure

Recommended file structure:

internsetu-ai/

- frontend/
- src/
  - pages/
  - components/
  - services/
  - utils/
  - data/
- backend/
- app/
  - models/
  - schemas/
  - routes/
  - services/
  - ml/
  - utils/
  - seed/
- docs/
- architecture.md
- matching-algorithm.md
- api-documentation.md
- database-schema.md
- data/
- candidates.json
- internships.json
- README.md

---

21. Success Metrics

The project will be considered successful if:

- Candidate can register and complete profile.
- Company can create internship.
- Matching engine generates relevant internship recommendations.
- Match score is visible.
- Explanation is visible.
- Candidate can apply.
- Company can shortlist/select/reject.
- Admin can view analytics.
- Admin can view fairness-related dashboard.
- Audit logs are generated.
- Project runs locally without errors.
- Demo flow is complete.

---

22. Risks and Mitigation

Risk 1: Matching score may look unrealistic

Mitigation:

- Use simple and explainable formula.
- Show score breakdown.
- Use sample data that clearly demonstrates good matching.

Risk 2: Project becomes too large

Mitigation:

- Focus on MVP.
- Build only 3 dashboards with core features.
- Avoid unnecessary features like chat, payment, or advanced verification in first version.

Risk 3: Fairness logic may be misunderstood

Mitigation:

- Present fairness score as policy-aware support.
- Do not say it replaces merit.
- Explain that it helps monitor representation.

Risk 4: Backend/frontend integration issues

Mitigation:

- Test APIs using Swagger/Postman first.
- Use simple JSON response format.
- Connect one dashboard at a time.

Risk 5: Time shortage

Mitigation:

- First build static UI with sample data.
- Then connect backend.
- Then improve AI engine.
- Keep demo-ready fallback data.

---

23. Implementation Priority

Build in this order:

1. Project setup
2. Database schema
3. Sample data
4. Backend APIs
5. Matching engine
6. Candidate dashboard
7. Company dashboard
8. Admin dashboard
9. Integration
10. Testing
11. Deployment
12. Documentation

---

24. Final Demo Flow

The demo should follow this order:

Step 1: Candidate Journey

- Candidate logs in.
- Candidate completes profile.
- Candidate sees recommended internships.

- Candidate checks match score and explanation.
- Candidate applies.

Step 2: Company Journey

- Company logs in.
- Company creates internship.
- Company sees ranked candidates.
- Company shortlists/selects candidate.

Step 3: Admin Journey

- Admin logs in.
- Admin views total candidates, companies, internships.
- Admin checks district-wise allocation.
- Admin checks category-wise representation.
- Admin checks capacity utilization.
- Admin views audit logs.

---

25. Final Product Summary

InternSetu AI is a full-stack AI-based internship allocation system designed for the PM Internship Scheme. It solves the problem of large-scale candidate-internship matching by using a transparent, explainable, and fairness-aware matching engine.

The project contains three dashboards:

- Candidate Dashboard for internship recommendation
- Company Dashboard for candidate shortlisting
- Admin Dashboard for fairness and capacity monitoring

The system is suitable for hackathons, resume projects, and future government-scale expansion.

The strongest value of this project is:

“Not just internship search, but fair and explainable internship allocation.”